

FaceFind API Documentation

This document provides a comprehensive list of API endpoints for both the **Node.js Backend** and the **Python ML Service**.

Architecture Overview

- **Backend Service:** `http://localhost:5001/api`
 - **ML Service:** `http://localhost:8000` (Internal use, but accessible)
 - **Base Media URL:** `http://localhost:5001/uploads`
-

17 Events API (`/api/events`)

1. List All Events

- **Endpoint:** `GET /`
- **Description:** Returns a list of all events stored in the database.
- **Response:**

```
{
  "success": true,
  "events": [...]
}
```

2. Get Event by Code (Guest)

- **Endpoint:** `GET /code/:code`
- **Description:** Fetches an active event using its unique 8-character code. Used by guests after scanning a QR code.
- **Response:**

```
{
  "success": true,
  "event": { "name": "...", "eventCode": "...", ... }
}
```

3. Get Event by ID

- **Endpoint:** `GET /:id`
- **Description:** Fetches details of a specific event by its MongoDB ID.

4. Create New Event

- **Endpoint:** `POST /`
- **Body:**

```
{
  "name": "Wedding of Alex & Sam",
  "description": "...",
  "date": "2025-06-15",
}
```

```
"location": "Central Park, NY"
}
```

- **Description:** Creates an event, generates a unique code, and auto-generates a QR code.

5. Delete Event

- **Endpoint:** DELETE `/:id`
- **Description:** Deletes the event and all associated photos from the database and disk.

6. Toggle Event Status

- **Endpoint:** PATCH `/:id/toggle`
- **Description:** Activates or deactivates an event (inactive events cannot be accessed by guests).

Photos API (`/api/photos`)

1. Get Event Photos

- **Endpoint:** GET `/event/:eventId`
- **Description:** Returns all photos uploaded for a specific event.

2. Upload Photos (Batch)

- **Endpoint:** POST `/upload/:eventId`
- **Body:** multipart/form-data with field `photos` (multiple files).
- **Description:** Uploads up to 50 images at once. Triggers background ML processing for face detection.

3. Delete Photo

- **Endpoint:** DELETE `/:photoId`
- **Description:** Removes a single photo from the database and disk.

Match API (`/api/match`)

1. Find My Photos

- **Endpoint:** POST `/:eventId`
- **Body:** multipart/form-data with field `selfie` (single image).
- **Description:** Extracts face features from the selfie and compares them against all photos in the event.
- **Response:**

```
{
  "success": true,
  "message": "Found 5 photo(s) with you!",
  "matches": [
    { "_id": "...", "url": "...", "uploadedAt": "..." }
  ]
}
```

ML Service API (Port 8000)

1. Health Check

- **Endpoint:** GET /health
- **Description:** Checks if the ML service and models are loaded correctly.

2. Process Event Photo

- **Endpoint:** POST /process-photo
- **Body:** multipart/form-data (file: image).
- **Description:** Detects all faces in an image and returns 512-D embeddings for each.

3. Get Selfie Embedding

- **Endpoint:** POST /get-embedding
- **Body:** multipart/form-data (file: image).
- **Description:** Specifically for selfies. Detects the largest face and returns its embedding.

4. Batch Face Matching

- **Endpoint:** POST /match-faces
- **Body:**

```
{
  "selfie_embedding": [0.1, 0.2, ...],
  "photo_embeddings": [
    { "photo_id": "123", "embeddings": [[...], [...]] }
  ],
  "threshold": 0.45
}
```

- **Description:** Computes cosine similarity between the selfie and all faces in the event photos.